

Smart MCQ Solver: Project Report

Project Workspace: 23f2004343-t22026

Metric: Mean Average Precision at 3 (MAP@3)

Target Cutoff: 0.73

Final Scores: Model 2 (BERT) — 0.74397, Model 3 (Ensemble) — 0.74608

1. Abstract

This report covers my work on the Smart MCQ Solver competition, where the task is to answer multiple-choice questions with five options (A through E) and rank the top three most likely correct answers. The scoring metric is MAP@3, which rewards getting the right answer near the top of the ranked list, not just somewhere in the top three.

I built three separate models to tackle this problem. The first is a transformer I wrote from scratch in PyTorch, with no pretrained weights. The second is a fine-tuned BERT model using Hugging Face. The third is a feature-based ensemble that combines TF-IDF, BM25, and sentence embeddings, fed into LightGBM and XGBoost. Model 2 reached a Kaggle score of 0.74397, and Model 3 did slightly better at 0.74608. Both clear the 0.73 cutoff comfortably. All three models were trained and validated with experiment tracking in Weights & Biases.

2. Introduction

2.1 Problem Statement

The competition gives each question a prompt and five answer choices labeled A to E. Only one option is correct, but instead of just picking one answer, I need to rank my top three guesses in order. If the real answer is first in my ranking, I get full credit. Second place gets half credit, third gets a third of the credit, and if the right answer isn't in my top three, I get nothing for that question.

2.2 Project Objective

My goal was to build a working pipeline that clears the 0.73 MAP@3 cutoff, while also fulfilling the project's three-model requirement: one model built entirely from scratch, one that fine-tunes a pretrained model, and one that takes a different, more experimental approach. I also wanted every training run properly logged in W&B so the three approaches could be compared fairly on the same metrics.

2.3 How This Report Is Organized

The next section talks about the dataset and how I cleaned and prepared it. After that, I explain how I tokenized text differently for each model. Then I walk through the three model architectures one by one, followed by a comparison of how they performed. I close with what I learned, what was hard, and what I'd try next if I kept working on this.

3. Dataset and Preprocessing

3.1 Dataset Description

The training set has 200 questions, and the test set has 50. Each question comes with a prompt and five options, so that works out to 1,000 option-pairs for training and 250 for test. The training data has answer labels; the test data doesn't, since that's what I'm predicting.

3.2 What the Data Looks Like

Looking through the questions, prompts run anywhere from about 15 words up to 140 words, averaging around 48 words. A lot of them use technical or scientific language. The options vary a lot in length too — some are just a single word or a date, others are full sentences, averaging around 12 words. The answer key is well balanced: each of the five letters (A through E) shows up as the correct answer about 20% of the time, so there's no shortcut where guessing one letter more often would help.

3.3 Cleaning and Reshaping the Data

Before feeding anything into a model, I cleaned the text: collapsed extra whitespace, lowercased everything, and stripped out weird unicode characters. Any missing context field got replaced with an empty string instead of being left as `NaN`, because leaving it as `NaN` causes problems once you start vectorizing text.

I also converted the data from one row per question into one row per question-option pair. So a single question with five options becomes five separate rows, each one paired with a single option and a 0 or 1 label depending on whether that option is correct. This "long format" makes it much easier to score each option independently, which all three of my models rely on in one way or another.

3.4 Avoiding Vocabulary Gaps

For Model 3, which relies on TF-IDF, I had to be careful about vocabulary. If I fit the TF-IDF vectorizer only on the training text, any word that only shows up in the test set would just get dropped, since the vectorizer wouldn't know it exists. To avoid that, I built the TF-IDF vocabulary from the combined text of both the training and test prompts and options, so nothing gets missed at inference time.

4. Tokenization Strategy

Each of the three models needed a different way of turning text into numbers, since they're built very differently.

4.1 Model 1 — Custom Vocabulary

Since Model 1 doesn't use any pretrained tokenizer, I built one myself using Python's `Counter` class. I split text on whitespace and punctuation, counted the 10,000 most common words, and mapped each one to an integer ID. I reserved index 0 for padding and index 1 for unknown words that don't appear in the vocabulary.

4.2 Model 2 — BERT's WordPiece Tokenizer

For Model 2, I used the standard `bert-base-uncased` tokenizer, which breaks words into subword pieces (WordPiece). Each question becomes five separate sequences, one per option, formatted as `[CLS] prompt+context [SEP] option [SEP]`, and everything gets padded or truncated to 128 tokens.

4.3 Model 3 — TF-IDF and Sentence Embeddings

Model 3 doesn't use a single tokenizer — it uses several text representations at once. I built a word-level TF-IDF vectorizer (using pairs of words, not just single words), a character-level TF-IDF vectorizer (which is good at catching spelling variations and technical abbreviations), and I also ran text through `all-MiniLM-L6-v2` to get 384-dimensional sentence embeddings that capture meaning rather than just word overlap.

4.4 Why These Choices Make Sense

Model 1's word-level vocabulary keeps things transparent and fully under my control, which fits its purpose as a from-scratch baseline. Model 2's WordPiece tokenizer is what BERT was trained with, so using anything else would throw away the benefit of pretraining. Model 3 mixes lexical matching (TF-IDF) with semantic matching (MiniLM) because relying on word overlap alone misses cases where the correct option is phrased differently from the question.

5. Modeling and Experimentation

5.1 Model 1: Transformer Built From Scratch

Model 1 is a small transformer encoder I wrote myself using raw PyTorch, without any Hugging Face code. The pipeline goes: word embeddings (256 dimensions) → fixed sinusoidal positional encoding, since attention on its own doesn't know word order → two transformer encoder layers, each with 4 attention heads → a masked mean pooling step that averages the token representations while ignoring padding → a small classifier head (`Linear` → `GELU` → `Dropout` → `Linear`) that outputs one score per option.

The core building block is scaled dot-product attention:

In plain terms: each token asks a question (the query), compares it against what every other token is offering (the keys), and then blends together the values based on how relevant each one is. Dividing by the square root of the key dimension keeps the numbers from getting too large, which would otherwise squash the softmax and make gradients vanish. Multiple heads

let the model track several kinds of relationships between words at once, and their outputs get combined at the end.

I trained this with `BCEWithLogitsLoss` since every option gets scored independently as correct or not, using AdamW with a learning rate of 1e-3, gradient clipping at 1.0, and a cosine annealing schedule, for 5 epochs at batch size 32.

5.2 Model 2: Fine-Tuned BERT

Model 2 uses `bert-base-uncased`, loaded through Hugging Face's `AutoModelForMultipleChoice` class. This is a 12-layer, 110-million-parameter model, and it's built specifically for the "one question, several choices" setup — you feed it five prompt-option pairs and it returns one score per option.

The key difference from Model 1 is that BERT starts from weights that were already trained on a huge amount of text, and its attention layers can look both left and right in the sentence at once (rather than being restricted in one direction). It also uses learned position embeddings instead of the fixed sinusoidal ones from Model 1. For each option, the final answer score comes from the `[CLS]` token's representation.

I fine-tuned it with `CrossEntropyLoss`, since this is really a 5-way classification problem (pick which of 5 options is correct), using AdamW with a learning rate of 2e-5, weight decay of 0.01, and a linear warmup schedule. I capped training at 3 epochs and saved the checkpoint with the lowest validation loss rather than the last epoch, to avoid overfitting on a fairly small dataset.

5.3 Model 3: Feature Engineering and Tree Ensemble

Model 3 takes a completely different approach — instead of a neural network, it builds 27 numeric features per question-option pair and feeds them into gradient-boosted trees.

The features fall into four groups:

- **Lexical matching:** cosine similarity from word and character TF-IDF, BM25 retrieval score (raw and normalized), Jaccard word overlap, and the proportion of option words that also appear in the question.
- **Semantic similarity:** cosine similarity between MiniLM sentence embeddings of the question and the option.
- **Length statistics:** option length, prompt length, and their ratios, both in characters and word count.
- **Relative rank features:** for each of the similarity features above, I also compute how far that option's score is from the average score in its question group, and where it ranks (1st through 5th) among the five options for that question.

This last group turned out to matter a lot. Raw similarity scores can vary a lot from question to question depending on how long or technical the prompt is. But if I instead ask "is this

option the best-scoring one out of its five siblings," that comparison holds steady no matter the absolute numbers, and tree models pick up on that pattern much more easily.

I trained LightGBM (500 trees) and XGBoost (400 trees) using 5-fold GroupKFold cross-validation, where all five options from the same question always stay in the same fold — this stops the model from accidentally learning from its own sibling options. I then blended the two models' predictions, sweeping a weight parameter to find the best mix, which landed at 55% LightGBM and 45% XGBoost.

6. Performance and Comparison

6.1 Metrics Used

I tracked four numbers across all models: MAP@3 (the competition metric), binary cross-entropy loss (how well-calibrated the probability outputs are), accuracy (whether the top prediction is right), and macro F1-score (which treats both classes evenly, useful since each question has one correct option and four wrong ones).

6.2 Results Table

Metric	Model 1: Scratch	Model 2: Fine-Tuned BERT	Model 3: Feature Ensemble
MAP@3 (Kaggle)	0.5840 (validation only)	0.74397	0.74608
Validation Accuracy	0.4620	0.5980	0.6650
Macro F1	0.4410	0.5750	0.6480
Validation Loss	0.6120	0.4850	0.3920
Training Time (Kaggle T4)	~2.5 min	~6.0 min	~45 sec

6.3 What the Results Tell Me

Model 3 came out on top with 0.74608, just ahead of Model 2's 0.74397. Both comfortably beat the 0.73 cutoff. What's interesting is that Model 3 got there without any deep learning training loop at all — it leans on a fixed sentence embedder, lexical signals like BM25, and those relative-rank features, then lets gradient boosting do the rest. It also trained in under a minute, compared to several minutes for BERT.

Model 1 scored lower at 0.5840, which makes sense — it has a small vocabulary, no pretraining, and a fairly small dataset to learn from. Still, it did what it was meant to do: prove that I understand how a transformer works at the code level, since every part of it (embeddings, attention, positional encoding, pooling) was written from scratch rather than imported.

7. Conclusion and Future Work

7.1 What I Accomplished

I built and compared three genuinely different approaches to the same problem: a transformer built from raw PyTorch, a fine-tuned pretrained BERT model, and a feature-based ensemble. All three were tracked in W&B under the same project so their runs could be compared side by side. Model 2 and Model 3 both beat the 0.73 qualification score, with Model 3 coming out slightly ahead at 0.74608.

7.2 Challenges I Ran Into

A few things gave me trouble along the way. Getting the TF-IDF vocabulary right meant fitting it on both train and test text together, otherwise test-only words would just vanish. Making sure the five options from one question never got split across different validation folds took some care — I used GroupKFold specifically to prevent that kind of leakage. I also had to handle Kaggle versus local file paths carefully, since the same notebook needs to run in both places without breaking. And for W&B credentials, I set things up to check Kaggle Secrets first, then fall back to a local `.env` file, so no API key is ever hardcoded into the notebook.

7.3 What I'd Try Next

If I kept working on this, a few ideas stand out. I could try a bigger semantic embedding model, like `bge-large-en-v1.5`, as an extra feature column in Model 3. I could also switch the tree models from plain binary classification to a proper pairwise ranking objective, since the task is really about ranking, not just classifying. Finally, generating some synthetic practice questions with a local open-source model could help pad out the training set, which is fairly small at only 200 questions.

8. References

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv:1810.04805*.
2. Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 30*, 5998–6008.
3. Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS 30*, 3146–3154.
4. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *ACM SIGKDD*, 785–794.
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP*.
6. Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.

7. Biewald, L. (2020). Experiment tracking with Weights & Biases. Software available from wandb.com.